Little Learners project report

SE4041 Assignment 02 Part A

K. Thiveesan IT23204348 15 October 2026

Project overview

Little Learners is an iOS learning application for children aged three to six in Sri Lanka. It brings English, Sinhala and Tamil letter activities, numbers and shape matching into one interface. Adults create child profiles and can inspect progress in a parent area. The project combines SwiftUI, SwiftData, on-device Vision text recognition, speech synthesis and charts.

The purpose is to support short, guided practice in recognising characters, following their outlines and matching basic shapes. Each child has separate progress, stars, tracing attempts and badges. This avoids mixing siblings' records when a family shares one device. The app is a learning aid rather than a validated assessment of literacy or development.

Audience and scope

The primary users are young children who benefit from large controls, prominent characters and immediate feedback. Parents and teachers are secondary users who choose a profile, assist with language and sound settings, and review progress. These choices motivate a simple home screen, visual avatars and an adult-oriented dashboard behind a maths question.

The implementation contains letters, numbers from 1 to 10, six matching shapes and eight reward badges. The README describes 26 English letters, 14 Sinhala vowels and 12 Tamil vowels. The language of the interface is an app-wide setting. It is not a separate preference stored for each child.

Implementation and evaluation status

The repository contains the implemented app and automated test source. Nine supplied screenshots illustrate the profile picker, home, lessons, tracing and shapes. Source inspection confirms the architecture and feature paths described in this report. A current Xcode build, completed user acceptance study and measured performance results remain to be recorded. Test source is not evidence of a successful test run.

Item	Reference
Student	**K. Thiveesan**
Student number	**IT23204348**
Module	**SE4041 Mobile Application Design and Development**
Repository	**https://github.com/ktthiveesan-glitch/LittleLearners**
Source snapshot	**744e1d2375b95f5b6e1a14bd7e39b79166bbde3a**
Document date	**15 October 2026**

Requirements and implementation

Requirement	Implementation evidence	Evaluation needed
Purpose and audience	Early learning in three languages with adult oversight	Validate usefulness with parents or teachers
Three or more navigable screens	Onboarding, profile picker, Home, lessons, tracing, shapes, rewards and parent area	Record end-to-end navigation
Advanced UI and customization	Spring animations, matchedGeometryEffect, custom bouncyPress modifier, reusable cards and adaptive grids	Inspect on different screens and text sizes
Navigation and architecture	Typed Route enum, Router and NavigationStack; observable view models	Demonstrate back navigation and profile switching
Advanced persistence	SwiftData models, inverse relationships and cascade deletion	Relaunch and deletion tests
Emerging technology	Vision VNRecognizeTextRequest checks English and digits, with geometry fallback	Demonstrate accepted and incorrect input
Branding and accessibility	Shared theme, colour assets, VoiceOver labels, Dynamic Type layouts	AppIcon currently has only Contents.json; device review needed
Testing and performance	75 @Test declarations and 20 UI test methods in source	Attach current test results and performance measurements
Documentation and presentation	Report, guide, technical notes and demonstration plan	Complete evidence fields and prepare final recording

The assignment requires advanced persistence under the heading Data Persistence and Networking. Its detailed wording does not require an online service. Local SwiftData is therefore relevant to this criterion. The separate emerging-technology criterion is addressed through Vision; the extent and quality of the integration must be demonstrated during the viva.

Assignment weighting is 15% for Part A and 10% for Part B, making 25% in total. The rubric for each part has its own 100-point criteria. No predicted grade is claimed here.

Interface design decisions

The main menu groups activities into Letters, Numbers and Shapes, with Rewards and Parents as secondary destinations. Large cards reduce the precision required for young users to select a task. A greeting and avatar show which child is active, while the star counter gives feedback about accumulated activity.

HomeView uses a LazyVGrid with an adaptive minimum width. At accessibility text sizes, selected horizontal arrangements change to vertical stacks. Onboarding uses a scrollable form with a capped content width, which helps avoid overly stretched fields on iPad. These are implementation measures; actual fit at every device size still needs visual verification.

The shared Theme and reusable components make controls consistent. BouncyPress supplies press feedback, ConfettiView celebrates success and ShakeEffect provides retry feedback. RootView uses a shared animation namespace to connect the profile avatar to Home. These effects should remain brief enough to support the activity rather than distract from it.

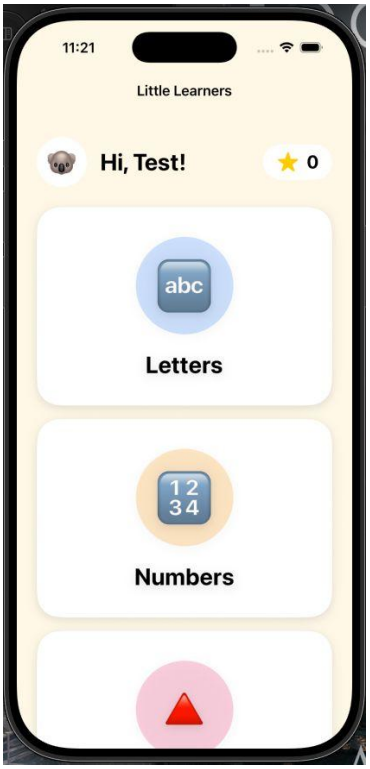


Figure 1. Home screen with active child and activity cards.

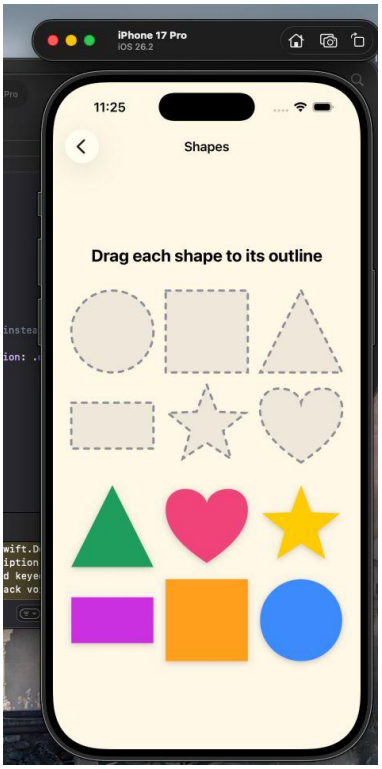


Figure 2. Shape matching activity with draggable pieces.

The screenshots establish the visible interface in the captured simulator session. They do not establish completion of accessibility, performance or persistence testing.

Architecture and data management

The app follows a feature-based MVVM structure. SwiftUI views display state and collect interaction. Observable view models manage feature behaviour. Services implement reusable work such as checking a tracing, speech, progress updates and badge awards. The typed Route enum gives navigation destinations a defined set of values rather than scattered string identifiers.

LittleLearnersApp creates a shared SwiftData container for ChildProfile, LessonProgress, TracingAttempt and EarnedBadge. RootView queries profiles, decides whether onboarding or profile selection is needed, and injects the active ChildProfile into the navigation environment. Selecting another child resets navigation to the root so a previous child's lesson screen does not remain active.

Model or storage	Responsibility
ChildProfile	Name, avatar, stars, creation date and stable profile identifier
LessonProgress	Completion records belonging to a child
TracingAttempt	Character, result, score and date linked to a child
EarnedBadge	A child's earned reward records
AppStorage	Selected profile identifier, app language and sound preferences

ChildProfile defines inverse relationships and cascade deletion for progress, attempts and badges. Deleting a child is intended to delete associated records. Parent-area reset is a different action: it clears progress while retaining the profile name and avatar. Both behaviours require persistence tests because they affect stored information.

ParentDashboardViewModel creates filtered fetch descriptors. Recent tracing is limited to ten records and sorted newest first. The stars chart loads the selected child's attempts from the beginning of the seven-day window. This avoids requesting every child's full tracing history for each dashboard display.

Reliability limitation

Several save paths use try? and do not present errors. In TracingViewModel, a successful drawing updates in-memory stars and progress before attempting persistence. A failed save could therefore look successful until a later launch. A production version should expose a recoverable save error and verify that the displayed result corresponds to durable storage.

Vision integration and tracing behaviour

A tracing is stored as strokes containing normalised coordinates. TracingViewModel converts these points to canvas coordinates, obtains a guide path and calls HandwritingRecognizer. Normalisation allows the stored drawing to remain proportional when the canvas size changes.

For English letters and numbers, the recognizer renders the drawing as black strokes on white. It first tries a rendered image containing three copies of the drawing, then a single copy. VNRecognizeTextRequest runs with accurate recognition, English recognition language and language correction disabled. Candidate text is compared with the expected character, including selected lookalike mappings.

When a candidate matches, the app returns a correct result with the candidate confidence. When no accepted match is found, it uses CoverageChecker. This fallback also applies when Vision recognises a different character; it is broader than a fallback only when Vision returns no text. The implementation therefore combines recognition with guide-following rather than providing a strict OCR correctness test.

For Sinhala and Tamil tracing, this implementation uses geometry directly. CoverageChecker samples points within the guide and compares them with a dense representation of the drawn strokes. A result passes when guide coverage is at least 70% and the share of drawing outside the guide is at most 40%. These thresholds are code constants, not measured recognition accuracy.

Meaningful contribution and limits

Vision gives the project a practical on-device checking path for English and digits. The geometry fallback helps support scripts outside the selected OCR path and allows guide-following practice without a server. However, covering a guide shape is not equivalent to recognising a well-formed character. A careful scribble or a confusing similar character can expose weaknesses that should be included in the test set.

A useful evaluation should include correct strokes, wrong letters, incomplete strokes, empty input and scribbles. For each, record the expected outcome and the observed outcome. The score is either a Vision confidence or a coverage fraction, so scores from these two paths should not be treated as the same kind of educational measurement.

Speech and localisation

SpeechService uses AVSpeechSynthesizer and caches voice availability. The listen control is available only when sound is enabled and the required voice exists. Sinhala speech availability may be limited on the selected device. Localisation uses a String Catalog, while a separate translation review file records content that needs native-speaker checking.

Testing and performance evaluation

The checked source contains 75 unit-test declarations in LittleLearnersTests and 20 test methods in LittleLearnersUITests. Test areas include profiles, badges, content, coverage, handwriting matching, localisation, lessons, parent-gate logic, parent-dashboard summaries, shapes and tracing. UI tests include onboarding, navigation, profile handling, language changes, the parental gate and launch performance.

The -uiTesting launch argument selects an in-memory database. The app also clears the relevant saved language, sound and active-profile settings for those test launches. This gives repeatable test setup and protects normal stored learning data.

Evidence	Current status	Submission action
Unit and UI test source	Present and inspected	Run in Xcode and attach the result summary
Current successful build	Not verified in this review	Record Xcode and simulator versions and build result
User acceptance testing	No completed results supplied	Use the workbook and report real observations
Performance	Optimisation techniques present; measured figures absent	Record launch time, memory and responsiveness
Screenshots	Nine supplied images inspected	Included in the illustrated guide
Screen recordings	Two files supplied	Check which app and feature sequence each covers before submission

Performance measures in the source include background Vision recognition, limiting recent-history fetches, filtering chart queries, caching speech voices and avoiding unnecessary tracing-screen updates for every stroke point. These choices describe intended efficiency. They do not prove a particular launch time, memory footprint or frame rate.

Results to complete before submission

Build and test date: [enter]. Xcode version: [enter]. Device or simulator and OS: [enter]. Unit tests passed / failed: [enter]. UI tests passed / failed: [enter]. Launch measurement and method: [enter]. Memory measurement and method: [enter]. User testing participants and consent procedure: [enter]. Evidence filenames: [enter].

If a check cannot be completed tonight, retain its status as not performed. Do not replace an absent result with a pass. Screenshots of test code do not substitute for executed test results.

Challenges and reflection

A central engineering challenge is that text recognition and tracing assessment answer different questions. Recognition attempts to identify a character; geometry assesses how closely strokes cover a guide. The current hybrid approach increases the range of practice activities, but it requires honest explanation of what a successful result means.

A second challenge is maintaining independent learning records on a shared device. The profile-centred model and filtered dashboard queries provide a clear separation. Resetting navigation during profile switching also helps keep the selected child consistent across the interface.

Multilingual design introduces content and device differences. Fonts must display different scripts, speech voices may not exist for every language, and interface translations need human review. Supporting a locale in code is only one part of making an activity understandable and appropriate for a child.

The work shows why completed implementation, successful execution and educational usefulness need separate evidence. A feature can exist in source without a fresh build result, and a responsive app can still need user feedback. The next development cycle should prioritise save-error handling, incorrect-drawing tests, native-speaker review and observed use with adults supporting the target age group.

Remaining limitations and future work

- **Add and verify an original app icon. The inspected AppIcon asset directory contains only its definition file.**
- **Complete device and accessibility checks, including large text, Dark Mode, VoiceOver navigation and rotation. Tracing still requires drawing input.**
- **Improve persistence error handling and test recovery after failed saves.**
- **Measure performance with reproducible device settings and retain results.**
- **Review all Sinhala and Tamil teaching content before wider use.**
- **Gather user feedback before claiming learning benefits or readiness for classroom adoption.**

References

SE4041 MADD Assignment 02, Semester 2 2026, pages 1 to 3 and 5 to 7. Supplied assignment brief.

LittleLearners repository, snapshot 744e1d2375b95f5b6e1a14bd7e39b79166bbde3a. Key files: README.md; App/RootView.swift; Models/ChildProfile.swift; Services/HandwritingRecognizer.swift; Services/CoverageChecker.swift; ViewModels/TracingViewModel.swift; ViewModels/ParentDashboardViewModel.swift; unit and UI test folders. https://github.com/ktthiveesan-glitch/LittleLearners

Nine application screenshots supplied by the student. AI assistance is disclosed in the accompanying record. The student should review the interpretation, add their own development reflection and verify all final evidence.